

Contagem sequencial e paralela de números primos

Matheus Marinho

28 de agosto de 2026

1. Objetivo

O objetivo desta tarefa foi implementar em C um programa que conta quantos números primos existem entre 2 e um limite `n`. Depois, o laço principal foi paralelizado diretamente com `#pragma omp parallel for`, sem modificar a lógica do contador. As versões sequencial e paralela foram comparadas quanto ao resultado e ao tempo de execução.

O experimento também introduz dois desafios da programação paralela: manter a correção quando várias threads alteram um mesmo dado e distribuir de forma equilibrada iterações que possuem custos diferentes.

2. Implementação

2.1 Teste de primalidade

A função `eh_primo` trata o número 2 separadamente, elimina os demais números pares e testa somente divisores ímpares. Os divisores são verificados até a raiz quadrada do número. A condição foi escrita como `divisor <= numero / divisor` para evitar uma multiplicação que poderia ultrapassar o limite do tipo `int`.

Não é necessário testar divisores maiores que a raiz quadrada: se um número composto possuir um fator maior que sua raiz, o fator correspondente será menor que a raiz e já terá sido testado.

2.2 Versão sequencial

Na versão sequencial, cada número entre 2 e `n` é testado e a variável `quantidade` é incrementada quando um primo é encontrado:

```
for (numero = 2; numero <= n; numero++) {
    if (eh_primo(numero)) {
        quantidade++;
    }
}
```

Como existe apenas uma thread, todos os incrementos são executados em sequência e o contador fornece o resultado de referência.

2.3 Versão paralela

A lógica original foi mantida e somente a diretiva solicitada foi acrescentada:

```
#pragma omp parallel for
for (numero = 2; numero <= n; numero++) {
    if (eh_primo(numero)) {
        quantidade++;
    }
}
```

As chamadas de `eh_primo` são independentes: descobrir se um número é primo não depende do resultado dos outros números. Entretanto, `quantidade` é compartilhada por todas as threads, o que cria um problema de correção discutido na Seção 5.

3. Método de medição

O programa foi compilado com:

```
gcc -O2 -Wall -Wextra -std=c99 -fopenmp primos.c -o primos.exe
```

Foi usado `n = 10.000.000` e a versão paralela executou com 20 threads, quantidade de processadores lógicos disponível no Intel Core i7-13650HX da máquina de teste. O tempo de parede foi medido por `QueryPerformanceCounter` no Windows e por `clock_gettime(CLOCK_MONOTONIC)` em sistemas POSIX. O programa completo foi executado cinco vezes pelo script `executar_testes.ps1`.

O uso de processos separados nas repetições permite observar variações no escalonamento e evita que uma única execução seja tratada como resultado absoluto. O valor central dos tempos, a mediana, foi usado no resumo.

4. Resultados

Execução	Primos sequencial	Tempo sequencial (s)	Primos paralelo	Tempo paralelo (s)	Speedup informado
1	664.579	2,294497	31.001	0,325218	7,055×
2	664.579	2,300537	31.001	0,306187	7,513×
3	664.579	2,259462	31.001	0,302109	7,479×
4	664.579	2,264279	31.001	0,302008	7,497×
5	664.579	2,259386	31.089	0,314836	7,176×
Mediana do tempo	664.579	2,264279	—	0,306187	7,395×

A versão sequencial sempre encontrou 664.579 primos, que é a contagem esperada até dez milhões. A versão paralela retornou 31.001 ou 31.089, dependendo da execução. Portanto, ela perdeu mais de 633 mil incrementos.

O tempo paralelo mediano foi menor, passando de 2,264279 s para 0,306187 s. A razão entre esses tempos é 7,395×. Contudo, esse valor não representa o *speedup* de uma solução correta: a execução paralela fez os testes de primalidade, mas falhou ao consolidar seus resultados. Desempenho só deve ser comparado como resultado útil depois que a equivalência das saídas for validada.

5. Desafio de correção

A expressão `quantidade++` parece uma única operação no código-fonte, mas envolve ler o valor atual, somar um e gravar o novo valor. Duas threads podem ler o mesmo valor antes que qualquer uma grave sua atualização. As duas então gravam o mesmo novo valor e um incremento é perdido.

Esse acesso concorrente sem sincronização é uma condição de corrida. Além dos incrementos perdidos observados, uma corrida de dados torna o comportamento do programa indefinido segundo o modelo de memória. A saída pode depender da ordem imprevisível de execução das threads e das decisões do compilador.

Por isso, obter resultados iguais em uma execução não provaria que o programa é correto. Da mesma forma, o fato de o problema aparecer claramente nesta máquina não garante que ele apareça com a mesma intensidade em outro ambiente.

Uma forma apropriada de corrigir o contador seria usar:

```
#pragma omp parallel for reduction(+:quantidade)
```

Com `reduction`, cada thread acumularia uma quantidade privada e o OpenMP somaria os valores ao final. Essa solução é apresentada somente como reflexão: ela não foi aplicada à versão medida, pois o objetivo deste experimento foi observar o que acontece ao acrescentar a diretiva sem alterar a lógica original.

6. Desafio de distribuição de carga

As iterações do laço não possuem custo uniforme. Há três casos principais:

- um número par é descartado quase imediatamente;
- um número composto ímpar termina quando seu primeiro divisor é encontrado;
- um número primo testa todos os divisores ímpares até sua raiz quadrada.

Além disso, a raiz quadrada cresce com o valor do candidato. Assim, as faixas próximas de `n` tendem a permitir mais testes que as faixas iniciais. Dependendo de como o runtime distribui os blocos, algumas threads podem terminar cedo e ficar ociosas enquanto outras continuam processando iterações mais caras. Como o laço só termina quando a última thread conclui seu trabalho, esse desequilíbrio limita o ganho de desempenho.

Um escalonamento dinâmico poderia redistribuir pequenos blocos às threads livres, por exemplo com `schedule(dynamic)`. Isso também introduziria overhead e não foi implementado, pois a tarefa solicita a observação e a explicação do problema, não sua solução. É possível que o desequilíbrio não apareça de maneira evidente nos tempos de uma execução; sua ausência visível não significa que todas as iterações tenham o mesmo custo.

7. Conclusão

A inclusão direta de `#pragma omp parallel for` reduziu o tempo observado, mas não produziu uma versão correta. O contador compartilhado sofreu uma condição de corrida e a resposta paralela variou entre execuções, enquanto a sequencial permaneceu em 664.579 primos.

O experimento mostra que paralelizar um laço não consiste apenas em dividi-lo entre threads. Primeiro é preciso verificar quais dados são independentes e quais são compartilhados. Depois, também é necessário considerar se as iterações têm custos semelhantes, pois uma divisão numericamente igual pode não representar uma divisão igual de trabalho.

Os dois problemas são potenciais e dependem da execução para se manifestarem no resultado ou no tempo. Mesmo quando uma medição não exibe erro ou grande desequilíbrio, a análise do código continua necessária para avaliar correção e desempenho.

8. Código-fonte

primos.c · 154 linhas

```
1  /*
2  * Tarefa 5 - Contagem sequencial e paralela de numeros primos
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c99 -fopenmp primos.c -o primos.exe
6  *
7  * Executar (argumentos opcionais: limite e numero de threads):
8  * ./primos.exe
9  * ./primos.exe 10000000 8
10 */
11
12 #define _POSIX_C_SOURCE 199309L
13
14 #include <errno.h>
15 #include <limits.h>
16 #include <omp.h>
17 #include <stdio.h>
18 #include <stdlib.h>
19
20 #ifdef _WIN32
21 #include <windows.h>
22 #else
23 #include <time.h>
24 #endif
25
26 static double walltime(void)
27 {
28 #ifdef _WIN32
29     LARGE_INTEGER contador;
30     LARGE_INTEGER frequencia;
31     QueryPerformanceCounter(&contador);
32     QueryPerformanceFrequency(&frequencia);
33     return (double)contador.QuadPart / (double)frequencia.QuadPart;
34 #else
35     struct timespec tempo;
36     clock_gettime(CLOCK_MONOTONIC, &tempo);
37     return tempo.tv_sec + tempo.tv_nsec / 1000000000.0;
38 #endif
39 }
40
41 static int eh_primo(int numero)
42 {
43     int divisor;
44
45     if (numero < 2) {
46         return 0;
47     }
48     if (numero == 2) {
49         return 1;
50     }
51     if (numero % 2 == 0) {
52         return 0;
53     }
54
55     for (divisor = 3; divisor <= numero / divisor; divisor += 2) {
56         if (numero % divisor == 0) {
57             return 0;
58         }
59     }
60     return 1;
61 }
62
63 static long long contar_sequencial(int n)
64 {
65     long long quantidade = 0;
66     int numero;
67
68     for (numero = 2; numero <= n; numero++) {
69         if (eh_primo(numero)) {
70             quantidade++;
71         }
72     }
73     return quantidade;
74 }
75
76 static long long contar_paralelo(int n)
77 {
78     long long quantidade = 0;
79     int numero;
80
81     /*
82     * O incremento continua compartilhado de proposito. A diretiva paraleliza
```

```

83     * o laço sem mudar sua logica e permite observar uma condicao de corrida.
84     */
85     #pragma omp parallel for
86     for (numero = 2; numero <= n; numero++) {
87         if (eh_primo(numero)) {
88             quantidade++;
89         }
90     }
91     return quantidade;
92 }
93
94 static int ler_inteiro_positivo(const char *texto, const char *nome)
95 {
96     char *fim;
97     long valor;
98
99     errno = 0;
100    valor = strtol(texto, &fim, 10);
101    if (errno != 0 || *texto == '\\0' || *fim != '\\0' ||
102        valor <= 0 || valor > INT_MAX) {
103        fprintf(stderr, "Valor invalido para %s: %s\\n", nome, texto);
104        exit(EXIT_FAILURE);
105    }
106    return (int)valor;
107 }
108
109 int main(int argc, char **argv)
110 {
111     int n = 10000000;
112     int threads;
113     long long resultado_sequencial;
114     long long resultado_paralelo;
115     double inicio;
116     double tempo_sequencial;
117     double tempo_paralelo;
118
119     if (argc > 3) {
120         fprintf(stderr, "Uso: %s [limite] [threads]\\n", argv[0]);
121         return EXIT_FAILURE;
122     }
123     if (argc >= 2) {
124         n = ler_inteiro_positivo(argv[1], "limite");
125     }
126
127     threads = omp_get_num_procs();
128     if (argc == 3) {
129         threads = ler_inteiro_positivo(argv[2], "threads");
130     }
131     omp_set_dynamic(0);
132     omp_set_num_threads(threads);
133
134     inicio = walltime();
135     resultado_sequencial = contar_sequencial(n);
136     tempo_sequencial = walltime() - inicio;
137
138     inicio = walltime();
139     resultado_paralelo = contar_paralelo(n);
140     tempo_paralelo = walltime() - inicio;
141
142     printf("Contagem de primos entre 2 e %d\\n", n);
143     printf("Threads na versao paralela: %d\\n\\n", threads);
144     printf("Versao      Primos      Tempo (s)\\n");
145     printf("Sequencial   %-12lld %.6f\\n", resultado_sequencial,
146         tempo_sequencial);
147     printf("Paralela      %-12lld %.6f\\n", resultado_paralelo,
148         tempo_paralelo);
149     printf("\\nSpeedup: %.3fx\\n", tempo_sequencial / tempo_paralelo);
150     printf("Diferenca no resultado: %lld\\n",
151         resultado_sequencial - resultado_paralelo);
152
153     return EXIT_SUCCESS;
154 }

```